Practice Report: Personalized Product Recommendation System for an E-commerce Platform

Student: Wang Fang Industry Mentor: Liu Qiang Host Company: Hangzhou Yunshang E-commerce Technology Co., Ltd.

1. Background and Research Problem

The host company operates a mid-sized e-commerce platform with approximately 2.3 million monthly active users and a catalog of over 180,000 SKUs. Prior to this project, product recommendations on the homepage and product-detail pages were generated using a simple rule-based system (best-sellers within category), which the marketing team identified as a key factor behind a stagnant click-through rate (CTR) of 2.1% on recommendation widgets, well below the industry benchmark of 3.5-4.5% cited in the company's quarterly product review. This project's goal was to design and deploy a personalized recommendation system that leverages user behavioral data (views, cart-adds, purchases) to increase recommendation relevance and, ultimately, CTR and conversion.

2. Technical Route

The system was designed as a hybrid recommender combining two components:

- (1) Collaborative filtering: a matrix-factorization model (implicit ALS, via the 'implicit' Python library) trained on 14 months of historical interaction data (views weighted 1x, cart-adds 3x, purchases 5x), producing latent user and item embeddings of dimension 64.
- (2) Content-based re-ranking: a lightweight gradient-boosted model (LightGBM) that re-ranks the top-200 collaborative-filtering candidates using item metadata features (category, price bracket, brand, recency) and user-level features (average order value, category affinity vector), to address the cold-start problem for newly listed products.

The two-stage pipeline (candidate generation via ALS, followed by LightGBM re-ranking) was chosen specifically because pure collaborative filtering underperforms on items with fewer than 20 historical interactions, which accounted for 34% of the active catalog at the time of the study - a gap the content-based re-ranking stage was designed to close.

The system was implemented in Python, with model training orchestrated via Apache Airflow (daily retraining), embeddings served through a Redis-backed feature store, and final recommendations exposed via a low-latency FastAPI microservice with a p99 latency budget of 80ms.

3. Experiments and Results

Offline evaluation was conducted on a held-out 14-day window of interaction logs (approximately 4.8 million events), using Recall@20 and NDCG@20 as primary metrics. The hybrid model achieved Recall@20 of 0.187 and NDCG@20 of 0.142, compared to 0.098 and 0.071 respectively for the existing rule-based baseline - improvements of 91% and 100% respectively.

A two-week online A/B test was subsequently run on 15% of platform traffic (approximately 345,000 users in the treatment group). The treatment group showed a CTR of 3.4% on recommendation widgets, versus 2.2% in the control group (a 55% relative improvement), and a 12% increase in attributed revenue per session. The difference in CTR was statistically significant ($p < 0.001$, two-proportion z-test).

Latency monitoring during the A/B test showed a p99 response time of 76ms, within the 80ms budget, with 99.97% of requests served successfully during the test window.

4. Deployment and Feasibility Considerations

Full production rollout was planned in three phases: (1) a 15% traffic ramp for two weeks (the A/B test itself), (2) a 50% ramp for one week with continuous monitoring of CTR, latency, and error rate dashboards, and (3) full rollout, contingent on no regression in the monitored metrics. The team also defined a rollback plan: if p99 latency exceeded 150ms for more than 10 consecutive minutes, or CTR dropped below the control-group baseline, the system would automatically fall back to the rule-based recommender via a feature-flag toggle, without requiring a code deployment. Daily retraining was scoped to complete within a 3-hour nightly batch window to avoid overlapping with peak traffic hours.

5. Conclusions

The hybrid collaborative-filtering and content-based re-ranking system significantly outperformed the existing rule-based baseline on both offline ranking metrics and live business metrics (CTR and revenue per session), while meeting the platform's latency requirements. Future work includes extending the content-based features with product image embeddings and evaluating a two-tower neural retrieval model as a potential replacement for the ALS candidate-generation stage.

6. Innovation Points

- - A two-stage hybrid architecture specifically designed to address the cold-start problem identified in this company's catalog (34% of items with sparse interaction history), rather than adopting a generic single-model recommender.
- - An automated feature-flag-based rollback mechanism tied to real-time latency and CTR thresholds, allowing safe, gradual production rollout without manual intervention during off-hours.
- - A staged traffic-ramp deployment plan (15% -> 50% -> 100%) with explicit go/no-go criteria at each stage, developed in collaboration with the platform's SRE team.